UFRGS
UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL

Curso de Especialização

Inteligência Artificial para Engenheiros

Disciplina: **Algoritmos e Programação (APE)**

Aula: 02 - Tipos de Dados, Controle de Fluxo

IA PARA ENGENHEIROS
UFRGS

Professor: **Raphael Martins Brum**

2026

Da aula passada: tipos de dados primitivos em Python

- A linguagem conta com os seguintes tipos de dados **escalares** nativos**:**
  - **int:** representa números inteiros
  - **float:** representa números reais (em ponto flutuante)
  - **bool:** representa valores booleanos (verdadeiro/falso)
  - **str:** representa texto (strings)
  - **NoneType:** tipo nulo especial
- Também conta com os seguintes tipos de dados **vetoriais** nativos:
  - complex (números complexos), list (lista de elementos), tuple (tuplas), range (sequências), dict (hash ou mapa), set (conjunto), frozenset (conjunto ordenado), bytes (vetor de dados de 8 bits constante), bytearray (vetor de dados de 8 bits variável), memoryview*

- Tipos customizados: objetos/classes

Da aula passada: operadores e funções em Python

- Operadores
  - Matemáticos: `+, -, *, /, %, **, //`
  - Booleanos: **`or, and, not`**
  - Comparadores: `<, <=, >, >=, ==, !=, is, is not`
  - Operadores *bitwise*: `|, ^, &, <<, >>, ~x`

- Python possui uma série de funções pré-definidas (*simple statements*), tipicamente capazes de manipular os tipos primitivos escalares e vetoriais
- Algumas funções:
  - print(), input(), help(), exit(), type()

Exercícios:
“30 Days of Python”

“Day 1” - helloworld.py

```python
# Introduction
# Day 1 - 30DaysOfPython Challenge

print("Hello World!")   # print hello world

print(2 + 3)   # addition(+)
print(3 - 1)   # subtraction(-)
print(2 * 3)   # multiplication(*)
print(3 + 2)   # addition(+)
print(3 - 2)   # subtraction(-)
print(3 * 2)   # multiplication(*)
print(3 / 2)   # division(/)
print(3 ** 2)  # exponential(**)
print(3 % 2)   # modulus(%)
print(3 // 2)  # Floor division operator(//)

# Checking data types

print(type(10))                   # Int
print(type(3.14))                 # Float
print(type(1 + 3j))               # Complex
print(type('Asabeneh'))           # String
print(type([1, 2, 3]))            # List
print(type({'name': 'Asabeneh'}))  # Dictionary
print(type({9.8, 3.14, 2.7}))    # Tuple
```

“Day 1” - helloworld.py

```python
# Introduction
# Day 1 - 30DaysOfPython Challenge

print("Hello World!")   # print hello world

print(2 + 3)   # addition(+)
print(3 - 1)   # subtraction(-)
print(2 * 3)   # multiplication(*)
print(3 + 2)   # addition(+)
print(3 - 2)   # subtraction(-)
print(3 * 2)   # multiplication(*)
print(3 / 2)   # division(/)
print(3 ** 2)  # exponential(**)
print(3 % 2)   # modulus(%)
print(3 // 2)  # Floor division operator(//)

# Checking data types

print(type(10))                   # Int
print(type(3.14))                 # Float
print(type(1 + 3j))               # Complex
print(type('Asabeneh'))           # String
print(type([1, 2, 3]))            # List
print(type({'name': 'Asabeneh'}))  # Dictionary
print(type({9.8, 3.14, 2.7}))    # Tuple
```

```
Hello World!
5
2
6
5
1
6
1.5
9
1
1
<class 'int'>
<class 'float'>
<class complex'>
<class 'str'>
<class 'list'>
<class 'dict'>
<class 'set'>
```

“Day 1” - helloworld.py

```python
# Introduction
# Day 1 - 30DaysOfPython Challenge

print("Hello World!")    # print hello world

print(2 + 3)    # addition(+)
print(3 - 1)    # subtraction(-)
print(2 * 3)    # multiplication(*)
print(3 + 2)    # addition(+)
print(3 - 2)    # subtraction(-)
print(3 * 2)    # multiplication(*)
print(3 / 2)    # division(/)
print(3 ** 2)   # exponential(**)
print(3 % 2)    # modulus(%)
print(3 // 2)   # Floor division operator(//)

# Checking data types

print(type(10))                    # Int
print(type(3.14))                  # Float
print(type(1 + 3j))                # Complex
print(type('Asabeneh'))            # String
print(type([1, 2, 3]))             # List
print(type({'name': 'Asabeneh'}))  # Dictionary
print(type({9.8, 3.14, 2.7}))     # Tuple
```

Diversas chamadas à função print, com argumentos distintos.

“Day 1” - helloworld.py

```python
# Introduction
# Day 1 - 30DaysOfPython Challenge

print("Hello World!")   # print hello world

print(2 + 3)    # addition(+)
print(3 - 1)    # subtraction(-)
print(2 * 3)    # multiplication(*)
print(3 + 2)    # addition(+)
print(3 - 2)    # subtraction(-)
print(3 * 2)    # multiplication(*)
print(3 / 2)    # division(/)
print(3 ** 2)   # exponential(**)
print(3 % 2)    # modulus(%)
print(3 // 2)   # Floor division operator(//)

# Checking data types

print(type(10))                    # Int
print(type(3.14))                  # Float
print(type(1 + 3j))                # Complex
print(type('Asabeneh'))            # String
print(type([1, 2, 3]))             # List
print(type({'name': 'Asabeneh'}))  # Dictionary
print(type({9.8, 3.14, 2.7}))     # Tuple
```

- “Hello World” é o argumento da função print.
- 3*2 é uma expressão que envolve **operandos** e **operadores**

Funciona como uma função de dois argumentos: __mul__(3,2)
[ver *operator overload*]

“Day 1” - helloworld.py

```python
# Introduction
# Day 1 - 30DaysOfPython Challenge

print("Hello World!")   # print hello world

print(2 + 3)    # addition(+)
print(3 - 1)    # subtraction(-)
print(2 * 3)    # multiplication(*)
print(3 + 2)    # addition(+)
print(3 - 2)    # subtraction(-)
print(3 * 2)    # multiplication(*)
print(3 / 2)    # division(/)
print(3 ** 2)   # exponential(**)
print(3 % 2)    # modulus(%)
print(3 // 2)   # Floor division operator(//)

# Checking data types

print(type(10))                    # Int
print(type(3.14))                  # Float
print(type(1 + 3j))                # Complex
print(type('Asabeneh'))            # String
print(type([1, 2, 3]))             # List
print(type({'name': 'Asabeneh'}))  # Dictionary
print(type({9.8, 3.14, 2.7}))     # Tuple
```

- type()
  - Devolve um objeto da classe `type`, que contém a informação da classe do argumento
- print(type(...))
  - Imprime na tela o tipo do objeto (*overload* da função print da classe `type`)

“Day 2” - variables.py

```python
# Variables in Python

first_name = 'Asabeneh'
last_name = 'Yetayeh'
country = 'Finland'
city = 'Helsinki'
age = 250
is_married = True
skills = ['HTML', 'CSS', 'JS', 'React', 'Python']
person_info = {
    'firstname': 'Asabeneh',
    'lastname': 'Yetayeh',
    'country': 'Finland',
    'city': 'Helsinki'
}

# Printing the values stored in the variables

print('First name:', first_name)
print('First name length:', len(first_name))
print('Last name: ', last_name)
print('Last name length: ', len(last_name))
print('Country: ', country)
print('City: ', city)
print('Age: ', age)
(...)
```

- Atribuição (=)
  - à esquerda: rótulo que está recebendo o valor
  - à direita: valor que está sendo associado ao rótulo
  - NÃO é uma operação de igualdade no sentido matemático
- Criação dinâmica de rótulos (não necessita declaração)

“Day 2” - variables.py

```
# Variables in Python

first_name = 'Asabeneh'
last_name = 'Yetayeh'
country = 'Finland'
city = 'Helsinki'
age = 250
is_married = True
skills = ['HTML', 'CSS', 'JS', 'React', 'Python']
person_info = {
    'firstname': 'Asabeneh',
    'lastname': 'Yetayeh',
    'country': 'Finland',
    'city': 'Helsinki'
}

# Printing the values stored in the variables

print('First name:', first_name)
print('First name length:', len(first_name))
print('Last name: ', last_name)
print('Last name length: ', len(last_name))
print('Country: ', country)
print('City: ', city)
print('Age: ', age)
(...)
```

- *string* [constante]
- *int*
- *list*

Escopo

- Regra LEGB
  - Local (L): escopo da função que está sendo executada
  - Enclosing (E): escopo da função-mãe, no caso de funções aninhadas
  - Global (G): escopo de mais alto nível em um módulo ou programa
  - Built-in (B): nomes pré-definidos len(), print(), and True.

Funções lambda

https://www.datacamp.com/tutorial/python-lambda-functions

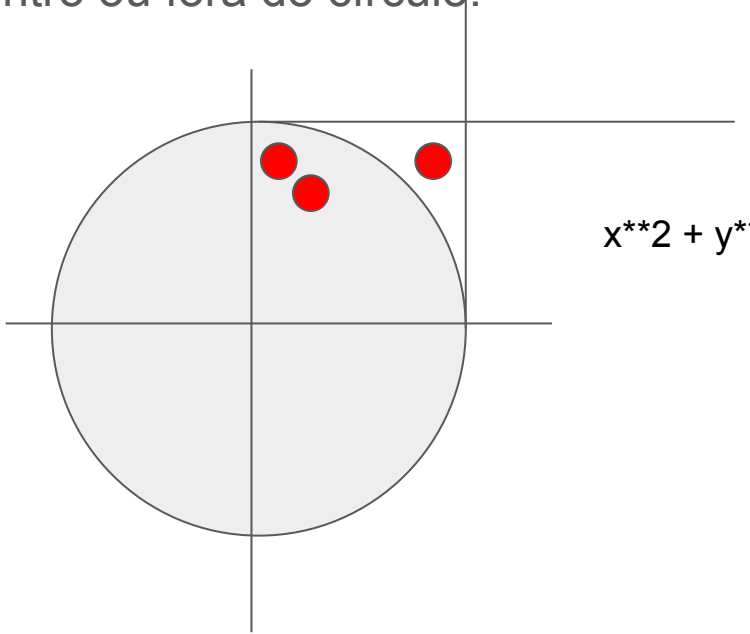
Exercício 1

1) Crie uma lista com 10 números quaisquer, diferentes, de 1 a 100.
2) Reverta (disponha os números na ordem inversa a ordem inicial) essa lista utilizando os métodos vistos até agora.

Exercício 2

Vamos calcular o Pi pelo método de Monte Carlo. Ideia do algoritmo: é “sortear” um ponto e verificar se ele cai dentro ou fora do círculo.

```
import random
pt = 0
pontos_dentro = 0
pontos = 100000
while pt < pontos :
  x = random.random()
  y = random.random()
  if ()
    pontos_dentro += 1
  pt += 1
```



x**2 + y**2 < 1

Entregas para a próxima semana

1) Algoritmo do Pi por Monte Carlo
   a) Incrementar o algoritmo feito em aula para realizar n rodadas de m pontos
   b) Guardar os resultados em uma lista.
   c) Calcular a média e o desvio padrão de Pi após as n rodadas, utilizando métodos próprios para as listas.

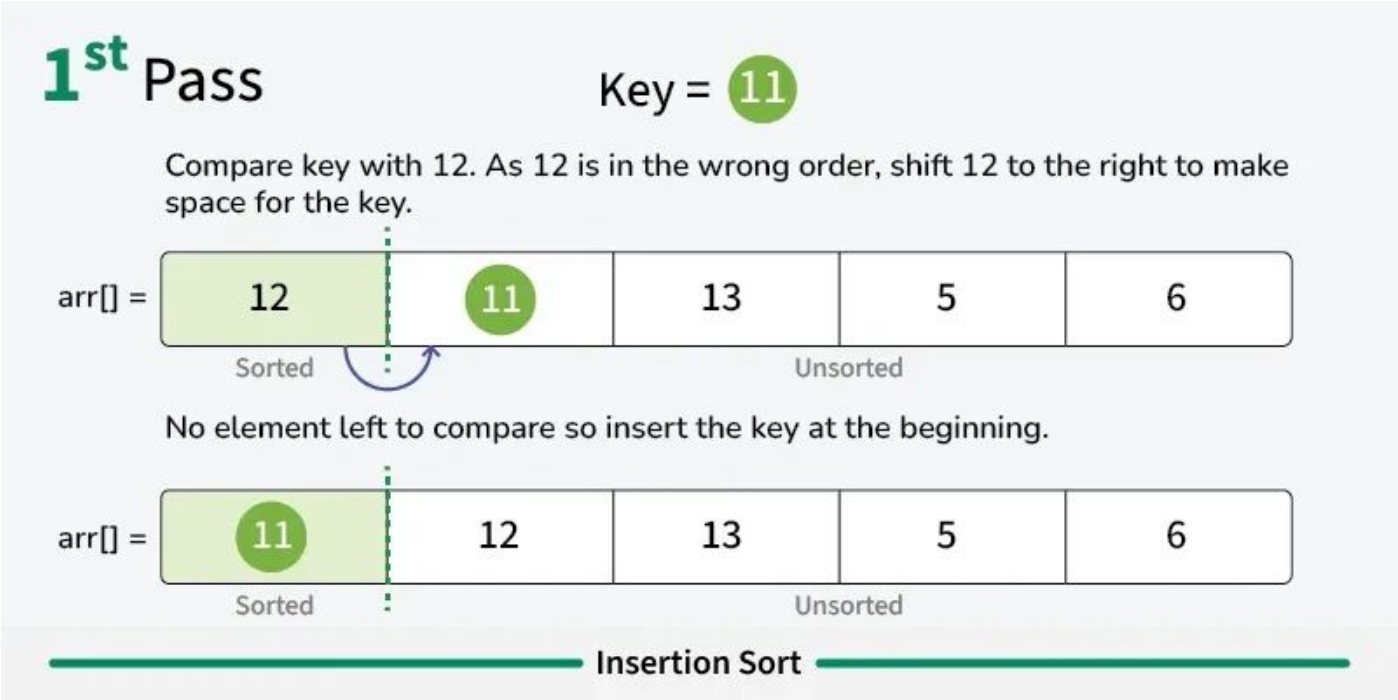
2) Algoritmos de ordenamento: Bubble Sort

3) Algoritmos de ordenamento: Insertion Sort

Aprimorar os programas para serem expressos em termos de funções

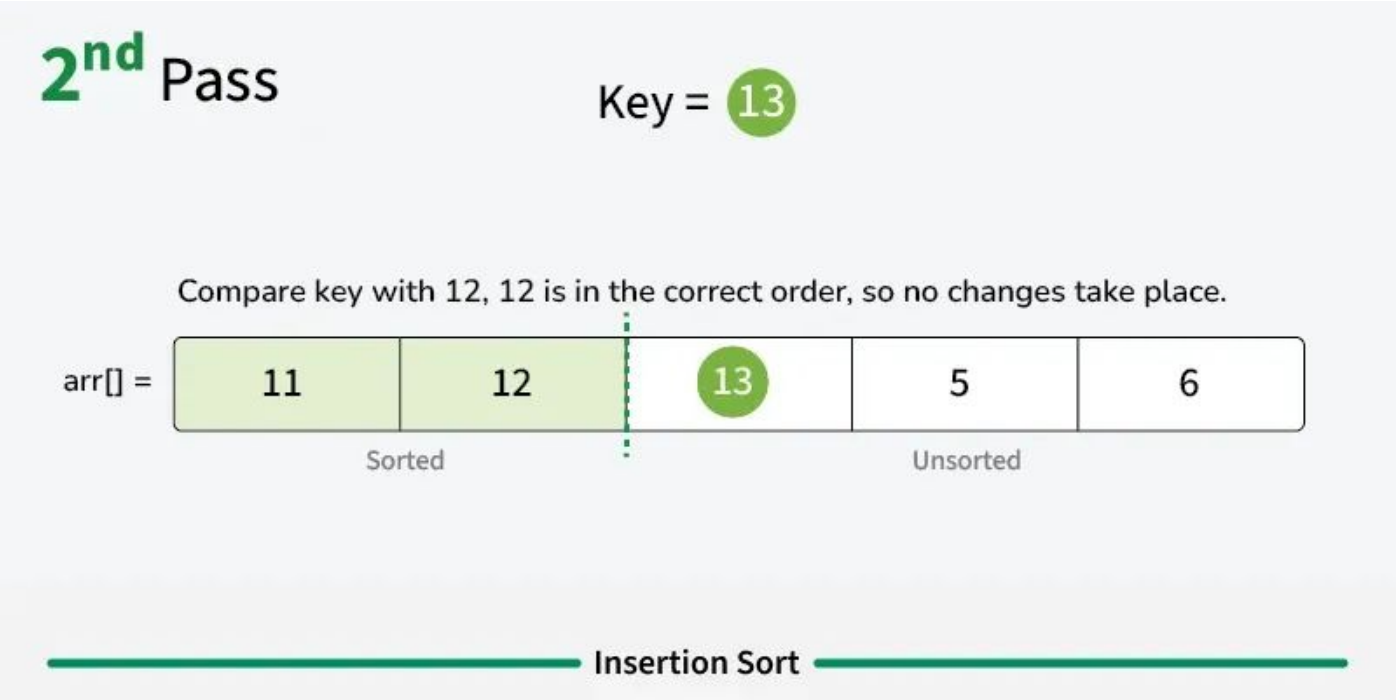
```
def funcao(arg1, arg2):
    [corpo da função]
funcao(var1, var2)
```

Insertion Sort

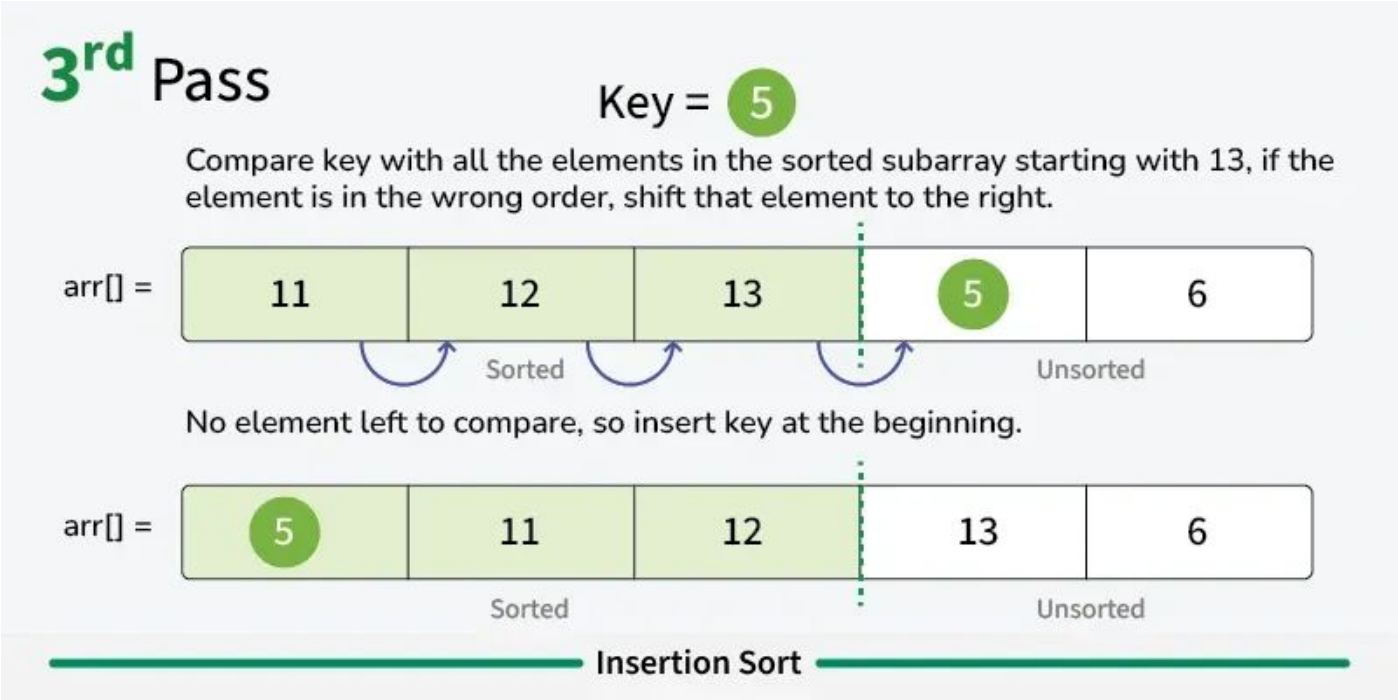


geeksforgeeks.org

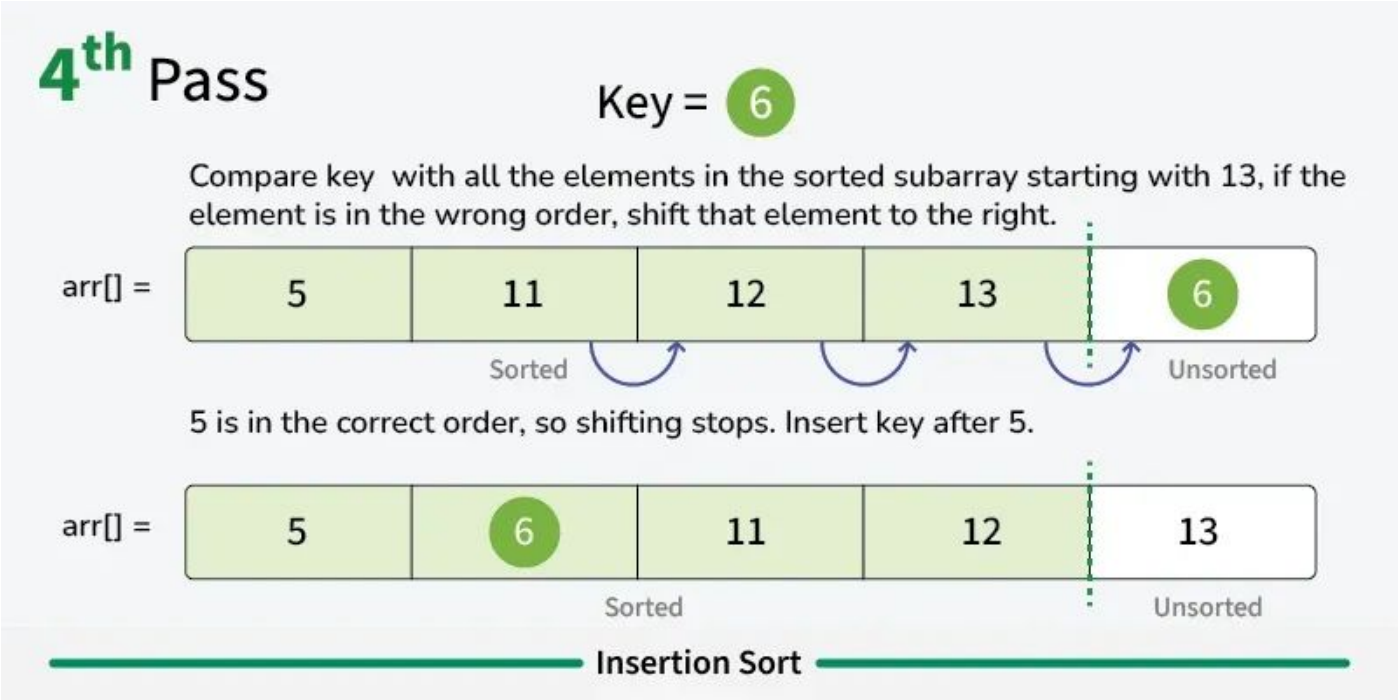
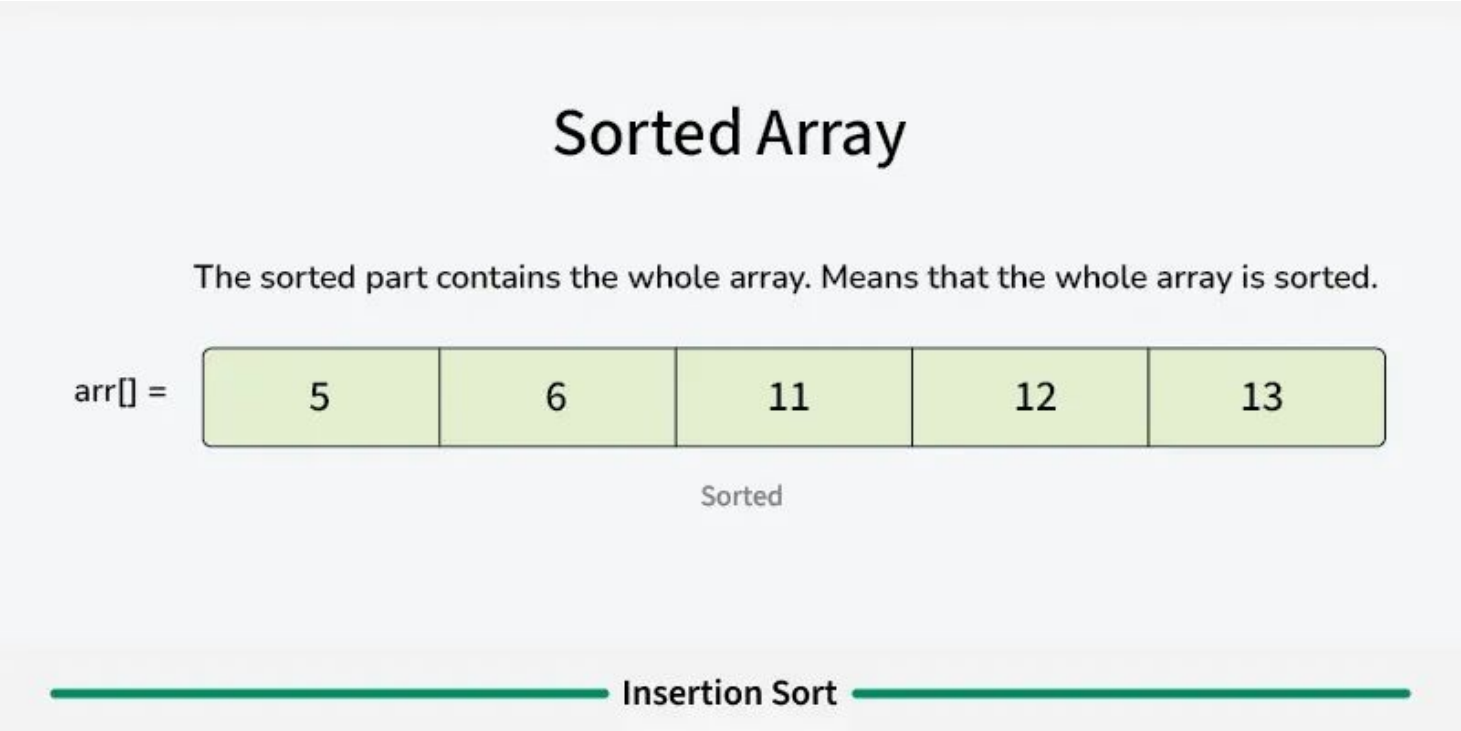
Insertion Sort



Insertion Sort



Insertion Sort



Insertion Sort